Отчет по выполнению
Индивидуального Домашнего Задания 2
Вариант 35

Сделано
Жарковой Татьяной Владимировной
студенткой 2 курса Программной инженерии
группы БПИ-253

Москва 2026
Высшая школа экономики

Оглавление

Аннотация 3

Формулировка задания 4

Общая структура решения 6

Архитектура многопоточности 6

Структура бригады 7

Состояния бригады 8

Команды потока бригады 9

Работа потока бригады 9

Синхронизация потоков 10

Передача команды бригаде 11

Основной поток и очередь событий 11

Завершение потоков 12

Обработка SIGINT 13

Остальной функционал 13

Обработка ошибок многопоточности 13

Дополнительные возможности программы 14

Аннотация

В данном отчете представлена работа индивидуального домашнего задания, вариант 35. Работа выполнена на языке C в личном репозитории.

Программа представляет собой событийную симуляцию работы диспетчерской службы. Она моделирует поступление экстренных вызовов, поиск подходящей свободной бригады, выезд бригады к месту происшествия, выполнение работ, завершение обслуживания и возвращение бригады на базу.

Симуляция выполняется последовательно по времени. Все происходящие в системе действия представлены в виде событий, помещаемых в очередь. На каждом шаге выбирается событие с минимальным временем наступления, текущее модельное время устанавливается равным времени события, после чего выполняется соответствующий обработчик.

Формулировка задания

В диспетчерскую службу поступают вызовы из разных районов города. Каждый вызов имеет уникальный номер, координаты, категорию срочности и требуемую специализацию бригады. Бригады различаются специализацией, скоростью движения и текущим местоположением. Диспетчер назначает свободную подходящую бригаду. После назначения бригада едет к месту вызова, оказывает помощь и затем становится доступной в текущем районе либо возвращается на базу. Срочность вызова может повышаться по мере ожидания. Один вызов не должен быть одновременно назначен нескольким бригадам, если условием явно не требуется совместная работа.

Требуется разработать консольное приложение, моделирующее работу службы экстренной помощи.

Входные параметры

• карта районов и время перемещения;
• количество и специализации бригад;
• число, время и место поступления вызовов;
• категории срочности;
• длительность оказания помощи;
• правила повышения приоритета;
• стратегия назначения и возврата бригад;
• максимальное время ожидания.

Отображаемые события

Необходимо отображать поступление вызова, изменение срочности, поиск и назначение бригады, выезд, прибытие, начало и окончание помощи, освобождение бригады, отмену и вызов, оставшийся без обслуживания.

Инварианты

• одна бригада обслуживает не более одного вызова;
• назначенная бригада соответствует требуемой специализации;
• один обычный вызов назначается не более чем одной бригаде;
• завершенный или отмененный вызов не назначается повторно;
• состояние доступности бригады изменяется согласованно с её маршрутом и работой.

Завершение

Моделирование завершается после окончания поступления вызовов и обработки всех возможных заявок либо по завершении смены с фиксацией необслуженных вызовов, либо по прерыванию пользователем, если задан режим неограниченного по времени выполнения.
Диспетчер, каждая бригада и каждый вызов являются самостоятельными участниками модели.

Общая структура решения

Программа реализована на языке C с использованием библиотеки POSIX Threads и представляет собой событийную симуляцию работы диспетчерской службы.

Программа моделирует поступление экстренных вызовов, поиск подходящей свободной бригады, выезд бригады к месту происшествия, выполнение работ, завершение обслуживания и возвращение бригады на базу.

В текущей реализации используется многопоточная архитектура. Главный поток отвечает за управление виртуальным временем и последовательную обработку очереди событий. Каждая аварийная бригада представлена отдельным рабочим потоком. Потоки бригад получают команды от диспетчера, изменяют состояние своей бригады и сообщают об окончании обработки команды.

Основные модули программы:

- `main.c` - запуск программы, обработка аргументов командной строки и завершение работы
- `config.c` - чтение и проверка конфигурационного файла
- `event_queue.c` - работа с очередью событий
- `simulation.c` - управление ходом симуляции
- `dispatch.c` - выбор и назначение бригад
- `event_handlers.c` - обработка событий
- `threads.c` - создание, работа и остановка потоков бригад
- `output.c` - вывод состояния системы и итоговой статистики
- `utils.c` - вспомогательные функции и проверки

Архитектура многопоточности

Основной сущностью, обладающей самостоятельным поведением, является аварийная бригада.

При запуске симуляции для каждой бригады создаётся отдельный поток `pthread`. Поток существует в течение всей симуляции и ожидает команды от диспетчера.

Главный поток управляет очередью событий и виртуальным временем, однако непосредственно не выполняет переходы состояния бригад.

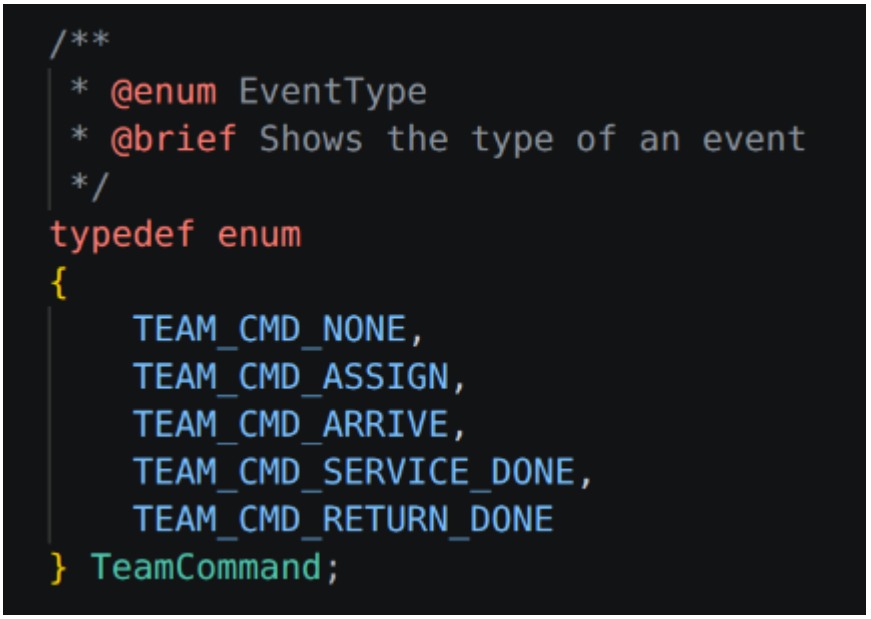
Когда событие требует изменения состояния конкретной бригады, главный поток передаёт ей команду. После этого соответствующий поток самостоятельно обрабатывает команду.

Таким образом, многопоточность используется непосредственно для моделирования поведения бригад, а не только для формального создания потоков.

Структура бригады

Для реализации взаимодействия потока бригады с диспетчером структура `Team` дополнена полями, связанными с межпоточной передачей команд.

Для команд используется `enum`:



В структуре бригады хранятся:

- TeamCommand pending_command - содержит команду, которую должен обработать поток бригады
- int command_district - содержит район, связанный с командой

Для свободной бригады `call_id` имеет значение`-1.

Состояния бригады

Бригада может находиться в одном из следующих состояний:

```
/**
 * @enum TeamState
 * @brief Shows the state of a team
 */
typedef enum
{
    TEAM_AVAILABLE,
    TEAM_EN_ROUTE,
    TEAM_AT_SCENE,
    TEAM_RETURNING

} TeamState;
```

Основной жизненный цикл бригады:

1. TEAM_AVAILABLE
2. TEAM_EN_ROUTE
3. TEAM_AT_SCENE
4. TEAM_RETURNING
5. TEAM_AVAILABLE

Если возврат на базу отключён, после завершения обслуживания бригада сразу переходит в состояние `TEAM_AVAILABLE`.

Команды потока бригады

Для взаимодействия между диспетчером и рабочими потоками используются четыре основные команды.

1. `TEAM_CMD_ASSIGN` - команда назначения вызова бригаде.

После получения команды бригада начинает движение к месту происшествия.

2. `TEAM_CMD_ARRIVE` - команда сообщает, что бригада прибыла в район вызова.

3. `TEAM_CMD_SERVICE_DONE` - команда сообщает об окончании обслуживания вызова.

После этого бригада становится либо возвращающейся на базу, либо сразу свободной.

4. `TEAM_CMD_RETURN_DONE` - команда сообщает об окончании возвращения на базу.

Работа потока бригады

Рабочий поток создаётся для каждой бригады.

После запуска поток не завершается сразу, а переходит в цикл ожидания команд.

Его работа выглядит следующим образом:

1. Создание `pthread`
2. Ожидание команды
3. Получение команды

4. Изменение состояния `Team`
5. Вывод результата
6. Сигнал об окончании обработки
7. Ожидание следующей команды

Поток ожидает семафор `assignment_sem`.

После получения сигнала он получает команду, считывает связанные параметры и выполняет соответствующий переход состояния.

После обработки команды поток сигнализирует главному потоку через `assignment_done_sem`.

Таким образом, главный поток может передать команду конкретной бригаде и дождаться подтверждения её обработки.

Синхронизация потоков

Для синхронизации используются:

- `pthread_mutex_t`
- `sem_t`
- `pthread_join().

Mutex используется для защиты общего состояния симуляции.

Семафоры используются для организации передачи команд между главным потоком и потоками бригад.

Для каждой бригады используются два основных семафора:

- assignment_sem - используется для передачи сигнала рабочему потоку о наличии новой команды
- assignment_done_sem - используется для уведомления главного потока о том, что команда была обработана

Передача команды бригаде

Передача команды реализована функцией:

```
int notify_team_thread(Simulation *sim, Team *team, TeamCommand
command, int district)
```

Алгоритм работы функции:

1. Проверить аргументы
2. Заблокировать `mutex`
3. Проверить состояние завершения
4. Записать команду
5. Записать район
6. Освободить `mutex`
7. `sem_post(assignment_sem)`
8. Ожидать `assignment_done_sem`
9. Команда обработана

При записи команды используется mutex, поэтому главный поток не может одновременно изменить данные команды с другим потоком.

После установки команды вызывается `sem_post()`, который разблокирует поток соответствующей бригады.

Основной поток и очередь событий

Основной поток сохраняет ответственность за управление виртуальным временем и очередью событий.

Очередь событий определяет строгий порядок наступления событий симуляции. Поэтому её обработка остаётся последовательной.
Таким образом, в программе используются два уровня выполнения:

- Главный поток

- Потоки бригад

Завершение потоков

Для корректного завершения программы используется общий флаг: `shutdown_requested`.

При завершении симуляции флаг устанавливается для всех потоков.

Но рабочий поток может находиться внутри:
`sem_wait(&team->assignment_sem)`

Поэтому одного изменения флага недостаточно.

Чтобы разблокировать поток, главный поток выполняет `sem_post()` для каждой бригады.

После выхода потоков из цикла выполняется: `pthread_join()` для каждого созданного потока.

Таким образом:

1. `shutdown_requested` = 1
2. `sem_post()` для всех `Team`
3. потоки просыпаются
4. проверяют `shutdown`
5. завершаются
6. `pthread_join()`

Это обеспечивает корректное завершение всех рабочих потоков перед завершением программы.

Обработка SIGINT

Программа поддерживает корректное завершение по `Ctrl+C`.

Обработчик сигнала устанавливает флаг остановки:
`static volatile sig_atomic_t stop_requested = 0`

При получении `SIGINT`:

1. `stop_requested` = 1
2. остановка симуляции
3. завершение потоков
4. `pthread_join()`
5. итоговая статистика

Остальной функционал

Остальной функционал описан в предыдущем отчёте.

Обработка ошибок многопоточности

Создание и завершение потоков, работа mutex и семафоров проверяются.

Особое внимание уделяется корректному завершению потоков, поскольку поток может находиться в ожидании семафора.

Все тесты, описанные прошлой реализацией, прошли проверки.

```
tay@zenbook:~/AKOS_homeworks_hse/hw_2_threads$ make test
[TEST] test_cancel ... PASS
[TEST] test_escalation ... PASS
[TEST] test_fastest ... PASS
[TEST] test_nearest ... PASS
[TEST] test_no_team ... PASS
[TEST] test_shift_limit ... PASS
[TEST] test_single_call ... PASS

Tests: 7 total, 7 passed, 0 failed
```

Дополнительные возможности программы

В программе реализованы следующие дополнительные возможности:

1. две стратегии выбора бригады — `NEAREST` и `FASTEST`
2. повышение приоритета ожидающих вызовов
3. автоматическая отмена вызовов
4. отдельный учёт необслуженных вызовов
5. возврат бригад на базу
6. возможность отключения возврата
7. визуальная задержка
8. отображение текущего состояния системы
9. сохранение истории в файл
10. итоговая статистика
11. обработка `SIGINT`
12. корректное завершение `pthread`
13. автоматический набор `тестов`
14. синхронизация потоков с использованием `mutex` и семафоров.